

AI Driven Anomaly Detection in Kubernetes Cluster

Project Team

Muhammad Hani	21I-2595
Muhammad Usman Azam	21I-0653
Daniyal Ahmed	21I-2493

Session 2021-2025

Supervised by

Dr. Muhammad Arshad Islam



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

September, 2024

Contents

1	Introduction	1
1.1	Problem Statement	2
1.2	Scope	2
1.3	Modules	3
1.3.1	Model Selection and Training Module	3
1.3.2	Network Setting Module	3
1.3.3	Traffic Security and YAML Auditing Module	3
1.3.4	Application Development Module	4
2	Project Requirements	5
2.1	Use Case Diagram	6
2.2	Use Cases	7
2.2.1	Monitoring Kubernetes Cluster	7
2.2.2	Automatically Detect Anomalies in Network Traffic	8
2.2.3	Log Aggregation and Event Tracing Using Tetragon	9
2.2.4	Automated Threat Mitigation	10
2.2.5	Scanning YAML Files for Misconfigurations	10
2.2.6	Writing Static Rules	11
2.2.7	Upload YAML File	11
2.2.8	View Security Alerts	12
2.2.9	Traffic Analysis	12
2.2.10	Access Admin Interface	13
2.3	Functional Requirements	13
2.3.1	Anomaly Detection	13
2.3.1.1	Monitoring Kubernetes Cluster Using Prometheus and Grafana	13
2.3.1.2	Network Traffic Management	13
2.3.1.3	Traffic Analysis through Deep Learning model	14
2.3.1.4	Automated Threat Mitigation	14
2.3.2	Event Tracing and Log Visualization with Tetragon	14
2.3.2.1	Tracing Events	14
2.3.2.2	Log Aggregation	14

2.3.3	Network Policies	15
2.3.3.1	Mitigation Using Network Policies	15
2.3.4	Honeypot for Traffic Analysis	15
2.3.4.1	Traffic Identification	15
2.3.4.2	Traffic Forwarding	15
2.3.4.3	Traffic Labeling	15
2.3.4.4	Detailed Analysis	15
2.3.5	Yaml File Checker	16
2.3.5.1	File Scanning	16
2.3.5.2	Error Identification	16
2.3.5.3	Send Notification	16
2.3.6	Web Application	16
2.3.6.1	File Upload Functionality	16
2.3.6.2	Misconfiguration Detection	16
2.3.6.3	Admin Panel	17
2.4	Non Functional Requirements	18
3	System Overview	19
3.1	Architectural Design	19
3.1.1	Process flow	20
3.2	Domain Model	20
3.3	Design Models	21
3.3.1	Activity Diagram	21
3.3.2	System Sequence Diagram	22
4	Implementation and Testing	33
4.1	Algorithm Design	33
4.1.1	Setting up Multi-Node Cluster	33
4.1.2	Traffic Mirroring	34
4.1.3	Data Preprocessing	35
4.1.4	Model Design	35
4.1.4.1	Model Architecture	35
4.2	Testing Details	36
4.2.1	Alerting System	36
4.2.1.1	Prometheus Alerting Overview	36
4.2.2	Grafana Alerting Overview	36
4.2.3	Tetragon Tracing Policies	37
4.2.3.1	Blocking mkdir command to restrict directory making	37
4.2.4	Yaml File Security Misconfiguration	38
4.2.5	Unit Testing	41
4.2.5.1	Data Preparation and Traffic Capture	41

4.2.5.2	Anomaly Detection and Alerting	43
4.2.5.3	Rate Limiting Application	45
4.2.5.4	Rate Limiting Reversion	47
5	Conclusions and Future Work	49
5.1	Conclusions	49
5.2	Future Work	50
	References	51

List of Figures

2.1	Use Case Diagram	6
3.1	Architecture Diagram	19
3.2	Domain Model	20
3.3	Activity Diagram	21
3.4	Monitoring Kubernetes Cluster	22
3.5	Automatically Detect Anomalies in Network Traffic	23
3.6	Automated Threat Mitigation	23
3.7	Scanning Yaml Files for Misconfigurations	24
3.8	Upload Yaml File	25
3.9	Writing Static Rules	26
3.10	View Security Alert	27
3.11	Traffic Analysis	28
3.12	Access Admin Interface	29
3.13	Log Aggregation and Event Tracing using Tetragon	30
3.14	Data Flow Level 0	30
3.15	Data Flow Level 1	31
4.1	Yaml file for Multi Node Cluster	33
4.2	Traffic Mirroring Algorithm	34
4.3	Data Preprocessing	35
4.4	Grafana Slack Alert	37
4.5	Block mkdir using Tetragon	38
4.6	Enter command to block	38
4.7	Security Smells in Yaml file	41
4.8	DoS attack dataset used in testing, showing sample data.	42
4.9	Traffic capture output from <code>server.py</code> , showing traffic flowing through NGINX.	43
4.10	Terminal showing that an anomaly was detected and a Slack alert was sent.	44
4.11	Slack alert confirming that the anomaly was detected.	44
4.12	Web interface showing multiple alerts from detected anomalies.	45
4.13	Web interface confirming that rate limiting was applied.	46
4.14	Locust report showing 91% of requests failed due to rate limiting.	46

4.15 Graph from Locust showing 138 out of 153 requests failed, confirming rate limiting.	47
4.16 Web interface confirming that the rate limit was removed.	47
4.17 Locust report showing 0% failures after the rate limit was removed.	48

List of Tables

Chapter 1

Introduction

Cyberattacks are becoming more frequent and sophisticated, which puts enterprises at serious risk of suffering major financial losses. An estimated 12.5 billion dollars were lost to cyberattacks in 2023[4]. Particularly in situations like Kubernetes clusters, where attack vectors can be broad and constantly growing, traditional methods cannot keep up with growing nature of cyber attacks. As attackers are evolving their methods to exploit vulnerabilities in the system, organization find it hard to secure their systems.

In particular, within the wider spectrum of cyber dangers, Distributed Denial of Service (DDoS)[2] attacks are a significant worry. The goal of DDoS attacks is to severely interrupt services by flooding systems with traffic. These assaults have the potential to cause significant financial harm; DDoS attack losses are estimated to cost companies millions of dollars worldwide each year. Attacker are now using botnets for DDos attacks which make it more of a challenge to mitigate it using traditional methods. Equally worrisome is the operational impact: DDoS attacks have the potential to completely destroy vital systems, resulting in protracted outages and a decline in customer confidence. Since businesses are depending more and more on digital services, it is essential to be able to identify and stop DDoS attacks in order to protect critical data and ensure company continuity.

Our project aims to address these challenges by developing an AI-based anomaly detection system tailored specifically for Kubernetes clusters. The system uses deep learning models, such as Transformer-based models [5] or Long Short Term Memory (LSTM) [6], to monitor network traffic and detect abnormal patterns that may indicate security risks. Because these are used to detect temporal trends in data, they are ideal for detecting long-term threats. Unlike traditional approaches, our AI-driven solution are able to recognize both known and unknown attack vectors by learning continuously, resulting in a more flexible and comprehensive protection mechanism. The system will adapt to evolving attacks by learning continuously from new attack patterns.

1.1 Problem Statement

Kubernetes[3], a widely adopted container orchestration platform, is an important component for deploying, managing, and scaling containerized applications with Docker. However, because of its widely adopted use, attackers target it for vulnerabilities. Many organizations find it difficult to efficiently monitor and analyze the massive amount of network traffic produced by Kubernetes clusters even with strong security measures. Modern cyber threats are sophisticated and can avoid detection by traditional detection methods, making it even more difficult to overcome. Kubernetes cluster is therefore still a prime target to a range of network anomalies, such as attempted intrusions, distributed denial-of-service (DDoS) attacks, and other malicious activities [1].

In addition, the increasing use of Kubernetes in different environments like cloud and standalone servers, has made its architecture more complex, thus increasing the range of attacks. The current anomaly detection tools of Kubernetes rely on static rules which makes attackers easier to bypass. This shows the need for a more intelligent solution which can detect patterns and behaviours.

1.2 Scope

- **In-Time Anomaly Detection:** KubeSecure will be monitoring and analysis the kubernetes clusters network traffic continuously to detect any unusual pattern which can be a potential harm for kubernetes security, such as DDos attacks.
- **AI-Driven Analysis:** KubeSecure will be using Deep Learning model to provide an adaptive and intelligent anomaly detection.
- **Automated Threat Mitigation:** KubeSecure will provide automated responses after detecting an anomaly like redirecting the traffic to Honeypots or rate limiting.
- **Static Rules Writing:** KubeSecure will be writing static network policy rules for cilium so that simple DDos attacks can be mitigated through predefined rules.
- **Testing and Validation:** KubeSecure will be testing our system by simulating various different attack scenarios using different computer system to ensure the effectiveness in real-world environments.

1.3 Modules

The suggested project is divided into several modules, each of which focuses on a different set of features that enhance the project's overall usability.

1.3.1 Model Selection and Training Module

In this module, we will setup Kubernetes cluster along with monitoring tools. Data pre-processing and Model training will begin.

1. Setup Kubernetes cluster with Grafana and Prometheus
2. Data Preprocessing and Deep Learning Model training

1.3.2 Network Setting Module

In this module we will setup Cilium and write Network policies. Training and testing of the selected model will be continued.

1. Setup Cilium
2. Network Policies in Cilium
3. Model Testing

1.3.3 Traffic Security and YAML Auditing Module

In this module we will integrate model in Kubernetes cluster to test it on streaming network traffic. We will also setup honeypots for traffic redirection and will make our yaml file misconfiguration tool.

1. Integrate model in Kubernetes
2. Setup TPOT Honeypot
3. Frontend website for yaml file misconfiguration
4. Backend logic for scanning yaml file

1.3.4 Application Development Module

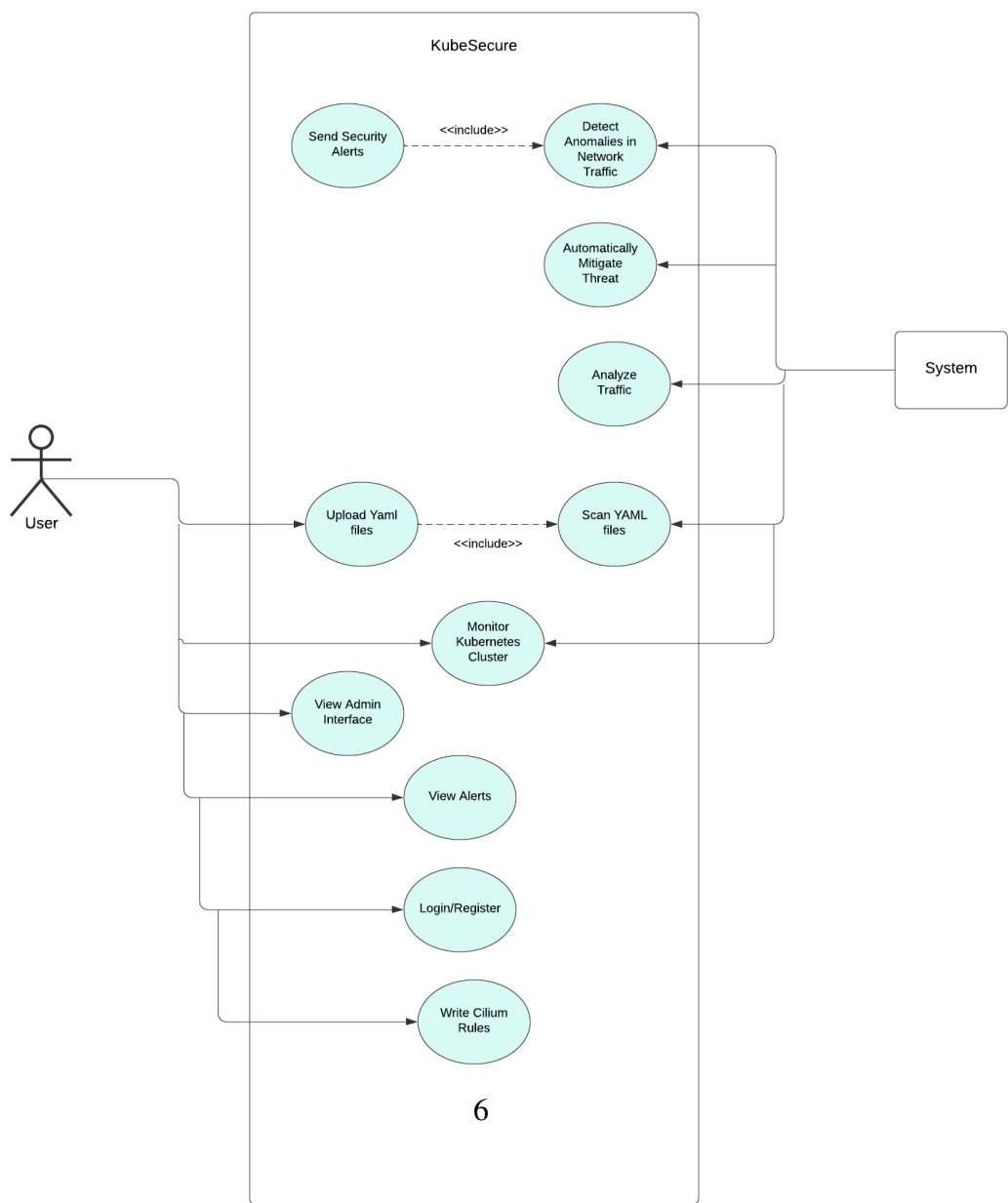
In this module we will develop android application for recieving alerts and will deploy our app on cloud.

1. Flutter App development
2. Deployment on Cloud

Chapter 2

Project Requirements

2.1 Use Case Diagram



2.2 Use Cases

2.2.1 Monitoring Kubernetes Cluster

Use Case	Monitoring Kubernetes Cluster	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, DevOps Engineers	
Precondition	The Kubernetes cluster is up and running, and Grafana and Prometheus are installed.	
Postcondition	Performance and anomaly measurements can be viewed and responded to.	
Main Success Scenario	User Actions	System Response
	1. User navigates to Grafana/Prometheus.	2. System opens Grafana/Prometheus interface.
	4. User chooses their desired dashboard to monitor the cluster.	3. System asks user to choose a dashboard to show.
	6. User monitors cluster through the dashboard.	5. System opens the selected dashboard.

2.2.2 Automatically Detect Anomalies in Network Traffic

Use Case	Automatically Detect Anomalies in Network Traffic	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, DevOps Engineers	
Precondition	The Kubernetes cluster is up and running, and the anomaly detection system is configured.	
Postcondition	Anomalies are detected and reported to the security team.	
Main Success Scenario	User Actions	System Response
	1. User initiates the monitoring process. 2. User reviews the detected anomalies.	3. System begins monitoring network traffic. 4. System detects anomalies based on configured parameters. 5. System alerts the user about the detected anomalies.

2.2.3 Log Aggregation and Event Tracing Using Tetragon

Use Case	Log Aggregation and Event Tracing using Tetragon	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, DevOps Engineers	
Precondition	Tetragon is in running condition and is present in the Kubernetes cluster, and event tracing and logging filters are configured.	
Postcondition	Logs are grouped and obtainable for searching events of your interest or excluding noisy events.	
Main Success Scenario	User Actions	System Response
	1. User configures filters for event tracing. 5. User reviews logs for any abnormalities.	2. System starts real-time monitoring of system calls, networks, and processes. 3. System groups the logs and stores them with a provision for accessing them as a single dataset. 4. System stores the grouped logs for centralized access.

2.2.4 Automated Threat Mitigation

Use Case	Automated Threat Mitigation	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, DevOps Engineers	
Precondition	The anomaly detection system must be active and configured to mitigate threats.	
Postcondition	Threats are mitigated automatically without manual intervention.	
Main Success Scenario	User Actions	System Response
	1. User configures mitigation policies.	3. System applies configured policies automatically.
	2. User enables automated threat mitigation.	4. System mitigates detected threats in in-time.
	6. User reviews logs of past mitigations.	5. System logs all mitigation actions for review.

2.2.5 Scanning YAML Files for Misconfigurations

Use Case	Scan YAML Files	
Scope	KubeSecure	
Level	High Level	
Primary Actor	DevOps Engineers	
Stakeholders	Security Team, DevOps Engineers	
Precondition	User has access to the YAML files for scanning.	
Postcondition	Vulnerabilities and misconfigurations are reported.	
Main Success Scenario	User Actions	System Response
	1. User uploads YAML files for scanning.	3. System scans YAML files for vulnerabilities.
	2. User initiates the scanning process.	4. System generates a report on findings.
	5. User reviews the scanning results.	6. System alerts the user of critical vulnerabilities.

2.2.6 Writing Static Rules

Use Case	Write Tetragon Rules	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Admin User	
Stakeholders	Security Team, Network Engineers	
Precondition	User must be authenticated and authorized to write rules.	
Postcondition	Tetragon rules are written and applied to the network policies.	
Main Success Scenario	User Actions	System Response
	1. User writes new/update Cilium rules.	2. System validates the syntax of the rules. 3. System applies the rules to the network policies. 4. System confirms the successful application of the rules.

2.2.7 Upload YAML File

Use Case	Upload YAML Files	
Scope	KubeSecure	
Level	High Level	
Primary Actor	DevOps Engineers	
Stakeholders	Security Team, DevOps Engineers	
Precondition	User is authenticated and has permissions to upload YAML files.	
Postcondition	YAML files are successfully uploaded and ready for scanning.	
Main Success Scenario	User Actions	System Response
	1. User navigates to the upload section. 2. User selects YAML files to upload. 3. User confirms the upload.	4. System accepts the YAML files. 5. System confirms successful upload. 6. System prepares the files for further processing.

2.2.8 View Security Alerts

Use Case	View Security Alerts	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, Incident Response Team	
Precondition	The alert system must be active and logging alerts.	
Postcondition	Security alerts are displayed for review.	
Main Success Scenario	User Actions	System Response
	1. User navigates to the alerts section. 4. User reviews the list of security alerts.	2. System displays the alerts highlighting critical alerts for immediate attention. 5. System logs user activity for audit.

2.2.9 Traffic Analysis

Use Case	Send Security Alert	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Security Team	
Stakeholders	Security Team, Incident Response Team	
Precondition	An alert has been generated and requires notification.	
Postcondition	The security alert is sent to designated recipients.	
Main Success Scenario	User Actions	System Response
	4. User navigates to Honeypot Dashboard. 5. User reviews Analysis Report.	1. System send the Traffic to Honeypot for Analysis. 2. Honeypot perform Analysis on the Traffic. 3. Honeypot generates Report on Analysis.

2.2.10 Access Admin Interface

Use Case	Access Admin Interface	
Scope	KubeSecure	
Level	High Level	
Primary Actor	Admin User	
Stakeholders	Security Team, Admins	
Precondition	User must be registered to access admin interface.	
Postcondition	Admin interface is displayed with all available options.	
Main Success Scenario	User Actions	System Response
	1. User enters their credentials. 4. User navigates through the available options. 5. User accesses specific functionalities like managing rules and monitoring dashboards.	2. System verifies user credentials and permissions. 3. System displays the admin interface with functionalities. 6. System allows access to the selected functionalities.

2.3 Functional Requirements

2.3.1 Anomaly Detection

2.3.1.1 Monitoring Kubernetes Cluster Using Prometheus and Grafana

- Network traffic metric will be collected from cilium
- Cluster and Application metrics will be collected from prometheus
- Grafana dashboards will show the metric collected from prometheus and cilium

2.3.1.2 Network Traffic Management

- The system will get streaming network traffic to a deployed application in Kubernetes Cluster.
- The incoming traffic will be duplicated to the Deep Learning model pod.
- Analysis will begin whenever the pod will get traffic.

2.3.1.3 Traffic Analysis through Deep Learning model

- Trained Deep Learning model pod will be deployed.
- It will get a traffic batch for analysis.
- When it will detect the traffic batch as malicious, it will issue an alert.

2.3.1.4 Automated Threat Mitigation

- When an alert will be issued by the model, prometheus will get the alert.
- Prometheus will call a web hook to run a python script to rate limit the incoming traffic.
- The python script will dynamically update cilium rules to rate limit or an EBPF script will be called.
- Alerts will be send to our Android application, slack and emails.

2.3.2 Event Tracing and Log Visualization with Tetragon

2.3.2.1 Tracing Events

- Tetragon will maintain and track actual event developments in the Kubernetes cluster at the moment.
- It will track system calls, network operations, and processes – that will help in illuminating the runtime picture.
- Filters will be additionally set to exclude all the events that are not related to security and compliance.
- Tetragon will provide detailed logs and event traces both interface based and terminal based.

2.3.2.2 Log Aggregation

- Tetragon will facilitate centrally situated log aggregation from the various nodes in a cluster.
- Logs will also contain information about the origin of the events found such as the pod, namespace or node.

2.3.3 Network Policies

2.3.3.1 Mitigation Using Network Policies

- The system will detect simple DoS attacks, such as TCP Flooding patterns by keeping track of violations of defined static rules.
- Upon recognizing a violation of defined rules, it will rate limit the traffic coming to the cluster

2.3.4 Honeypot for Traffic Analysis

2.3.4.1 Traffic Identification

- According to its specific criteria of suspicious packet loads, aberrant traffic dynamics, and common attack origins, the framework will highlight harmful traffic amassings.

2.3.4.2 Traffic Forwarding

- When it spots malicious traffic, the system will pass along that batch for thorough investigation in the TPOT Honeypot.

2.3.4.3 Traffic Labeling

- Prior to sending the threatening traffic directed at TPOT Honeypot, the system will feature metadata indicating the source IP, the time of detection.

2.3.4.4 Detailed Analysis

- The TPOT Honeypot will generate enhanced analysis reports that contribute information on the behaviors of traffic, attack trends, and practical exploit vectors, all as part of improving mitigation and security practices.

2.3.5 Yaml File Checker

2.3.5.1 File Scanning

The system shall scan uploaded Kubernetes .yaml files for common configuration errors, including but not limited to:

- The findings indicate that a number of configurations in RBAC (Role-Based Access Control) are either left unenabled or improperly configured.
- Resource allocations can suffer from a problem that makes them either inadequate or excessive.
- Errors result from poor security statements regarding pods, inadequate configuration of network settings, or the configuration of volumes.
- Conditioned to obsolete API versions or choosing setups that do not provide security.

2.3.5.2 Error Identification

- The system will separate out misconfigurations found in .yaml files by evaluating their content against industry standards and security policies.

2.3.5.3 Send Notification

- The platform will issue emails, dashboard notifications, or system signals over the issues it finds.

2.3.6 Web Application

2.3.6.1 File Upload Functionality

- This platform will support web submissions of Kubernetes .yaml files for analysis.

2.3.6.2 Misconfiguration Detection

- Upon the submission of a .yaml file, the system will rapidly start its processing and will comb through for misconfigurations.

2.3.6.3 Admin Panel

- Admin can view and setup cilium rules
- Drop down menu for selecting values for network policies

2.4 Non Functional Requirements

User-Friendly: The system will be optimized whereby one can learn it easily and operate without much difficulty.

In-Time Data Processing: In order to manage all these flows the system will be able to process in time data with minimal delay.

Scalability: KubeSecure is optimistically designed to be scalable, and this means that it can scale up to support the increased network traffic and monitoring needs or requirements.

Reliability: The system shall be highly reliable, with redundancy and failover mechanisms in place to ensure continuous operation even during high traffic or under attack.

Interoperability: The system shall be compatible with various existing tools and platforms used in Kubernetes environments, such as different CI/CD pipelines, monitoring systems, and security tools, allowing for seamless integration.

Security: The traffic analysis deep learning pod is to be isolated of external access so that changes (may) not be possible by unauthorized actors. Under high level DDoS attack the pod will remain secure and stable while continuously monitoring and detecting anomalies without compromise.

Chapter 3

System Overview

3.1 Architectural Design

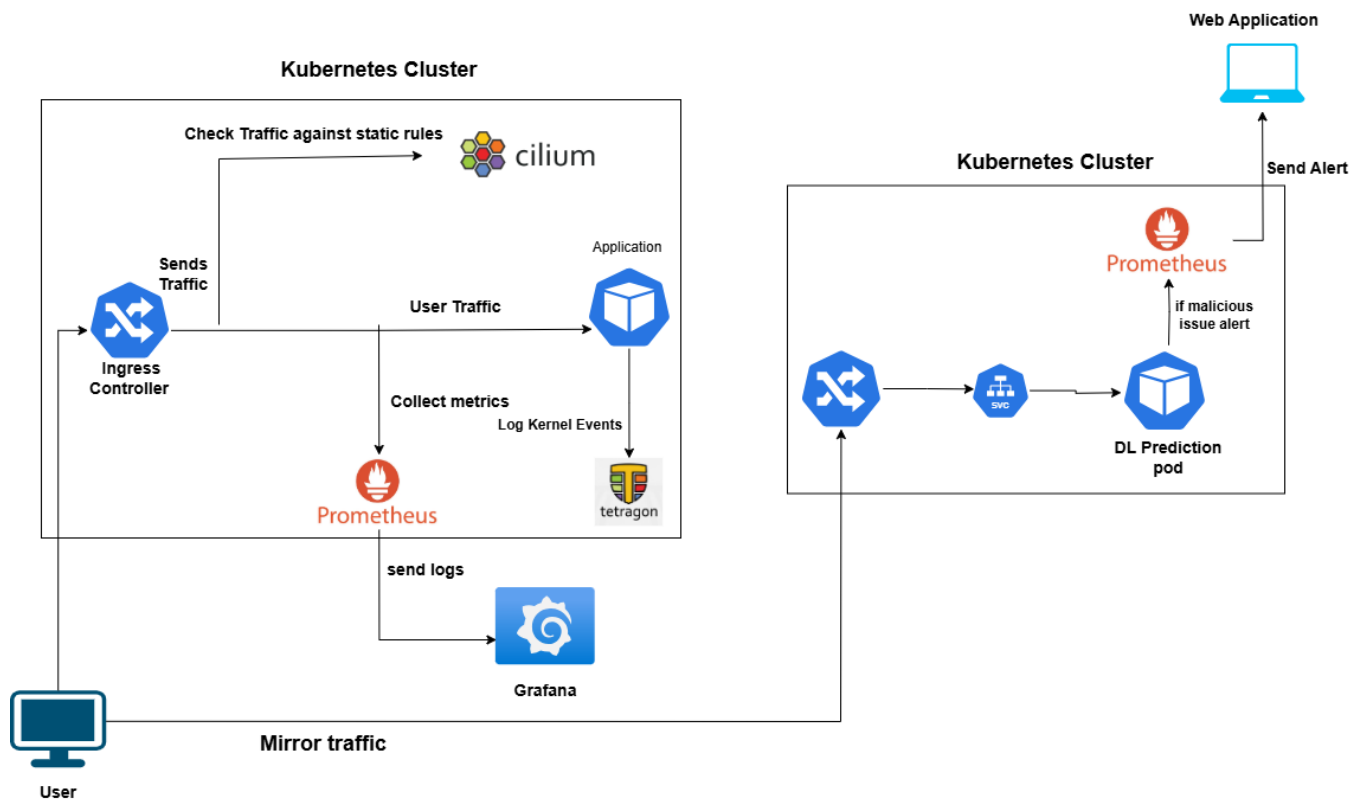


Figure 3.1: Architecture Diagram

3.1.1 Process flow

- User sends traffic to the application pod.
- The traffic goes to Kubernetes Ingress controller and it forwards traffic to application pod.
- It duplicates the traffic to Cilium and to Kubernetes Cluster which contains Deep Learning model for prediction.
- Cilium will check the traffic against predefined rules.
- If Cilium or Deep Learning model reports any threat. It will send alerts to our android application and the traffic batch will be given to honeypot for traffic analysis.
- Prometheus will actively collect logs and show to graphs in Grafana.

3.2 Domain Model

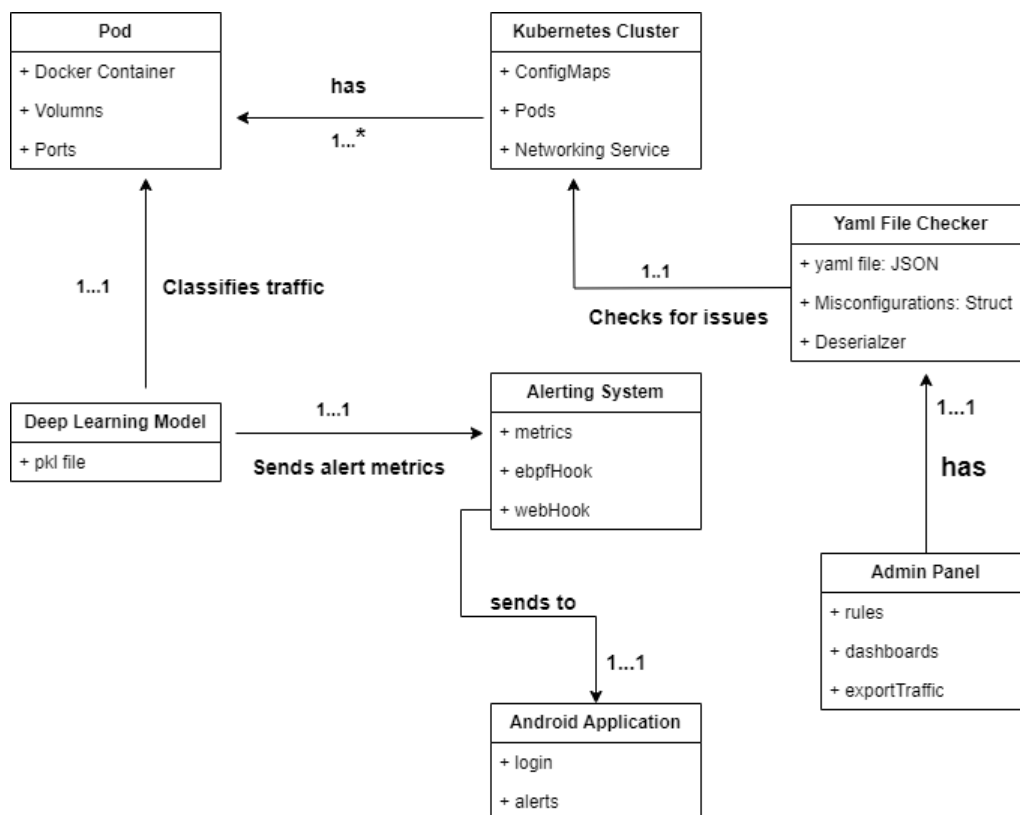


Figure 3.2: Domain Model

3.3.2 System Sequence Diagram

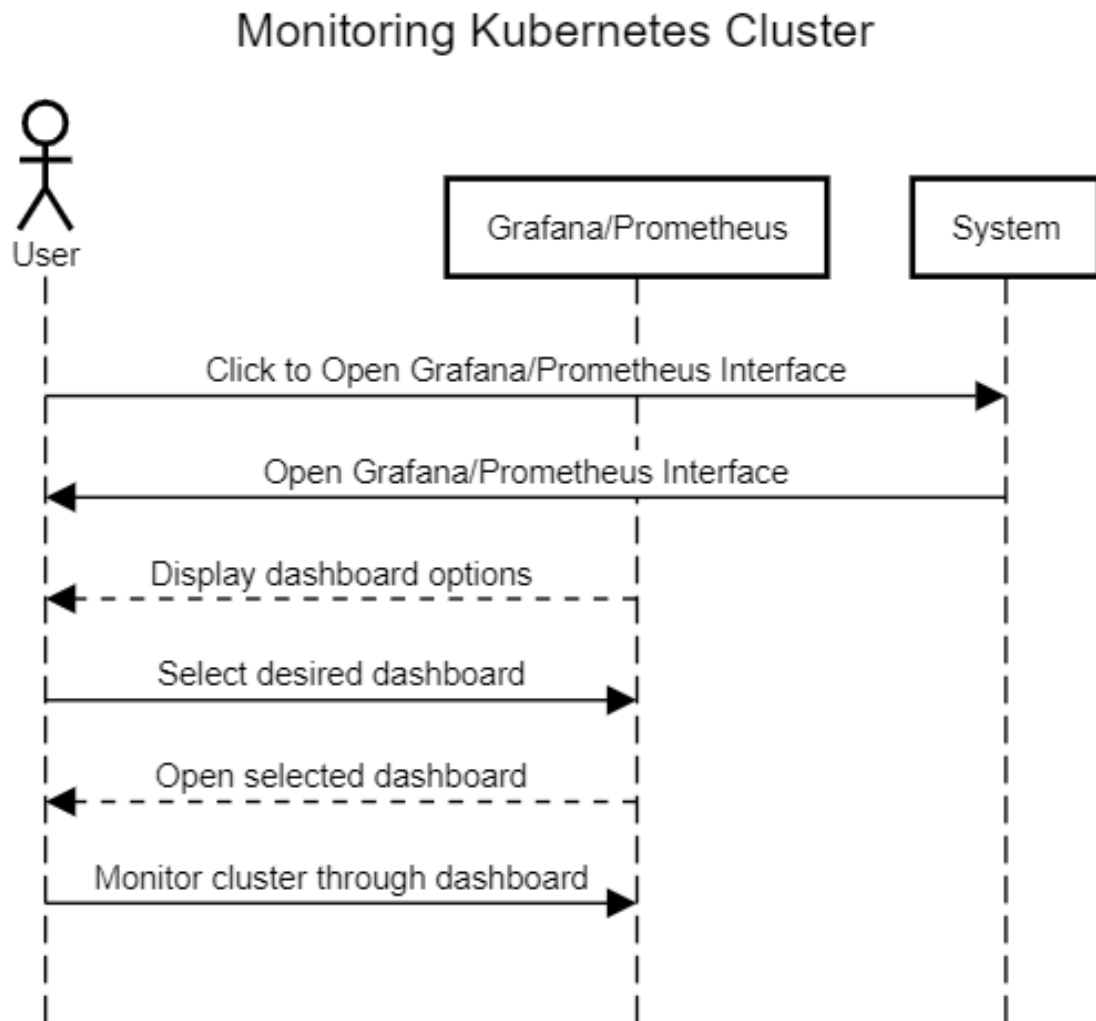


Figure 3.4: Monitoring Kubernetes Cluster

Automatically Detect Anomalies in Network Traffic

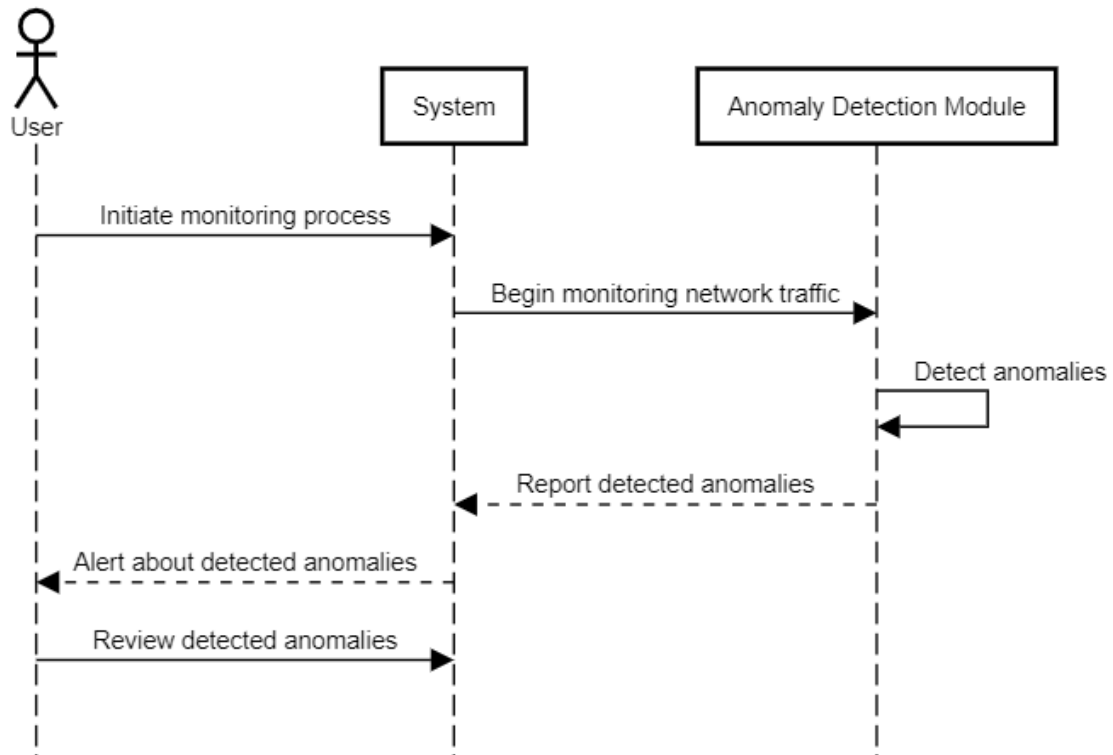


Figure 3.5: Automatically Detect Anomalies in Network Traffic

Automated Threat Mitigation

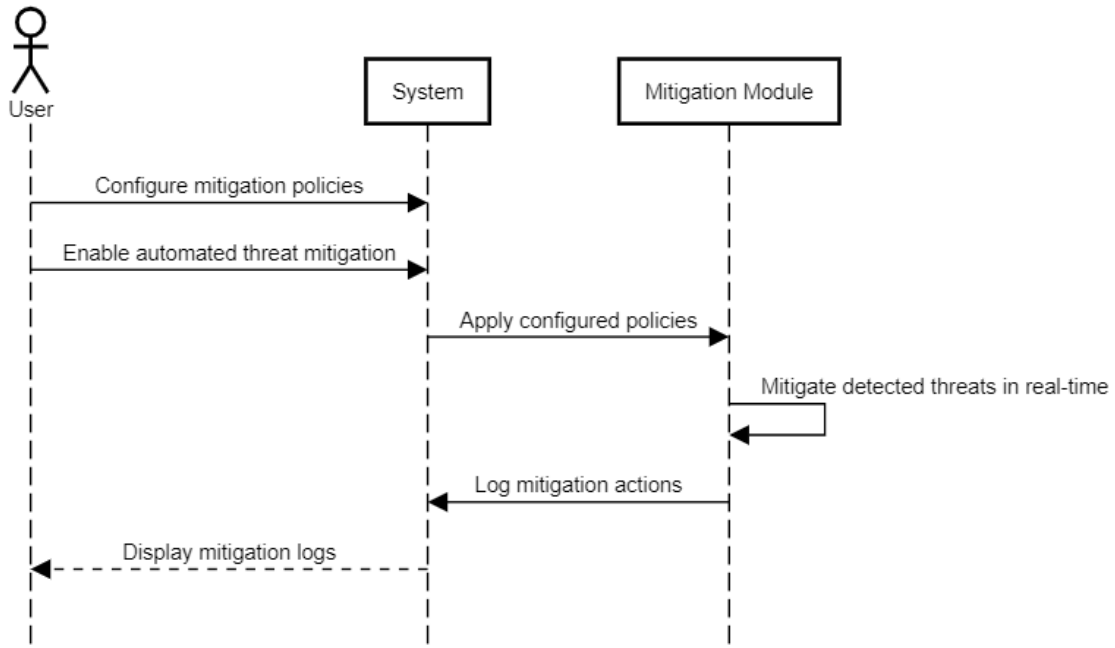


Figure 3.6: Automated Threat Mitigation

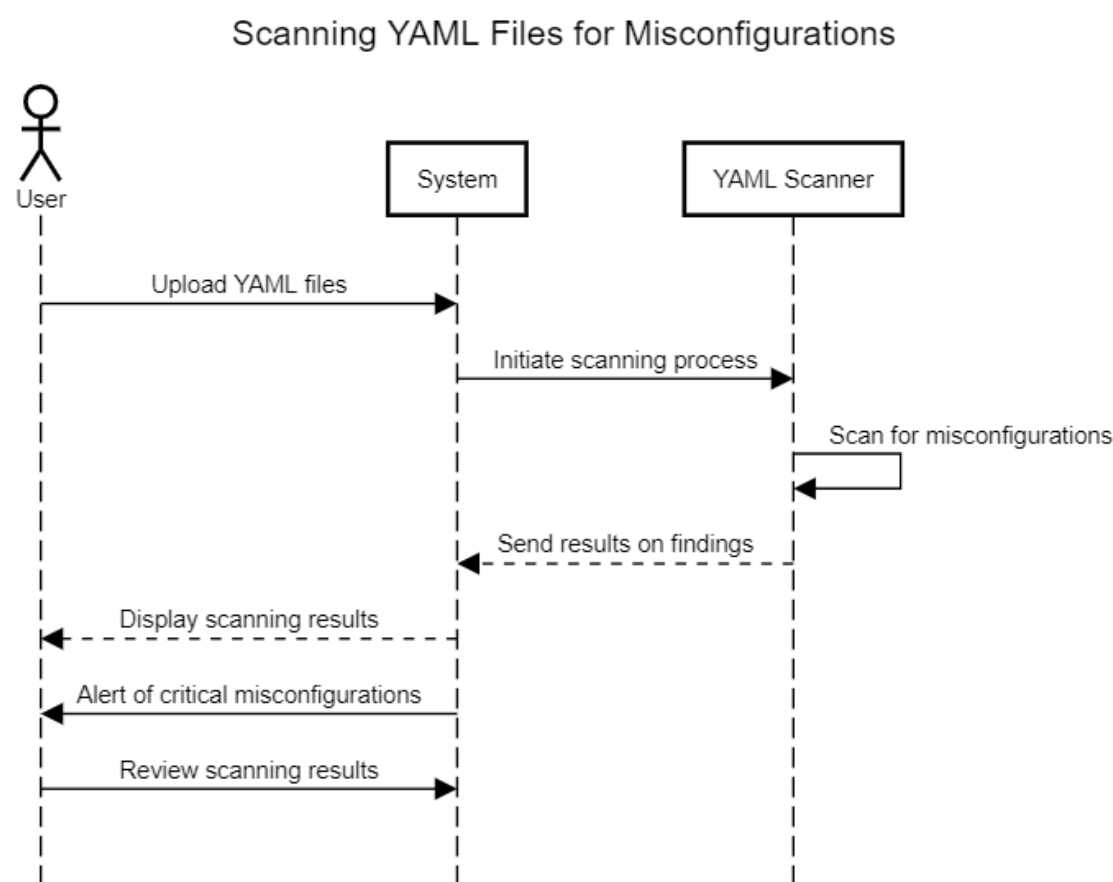


Figure 3.7: Scanning Yaml Files for Misconfigurations

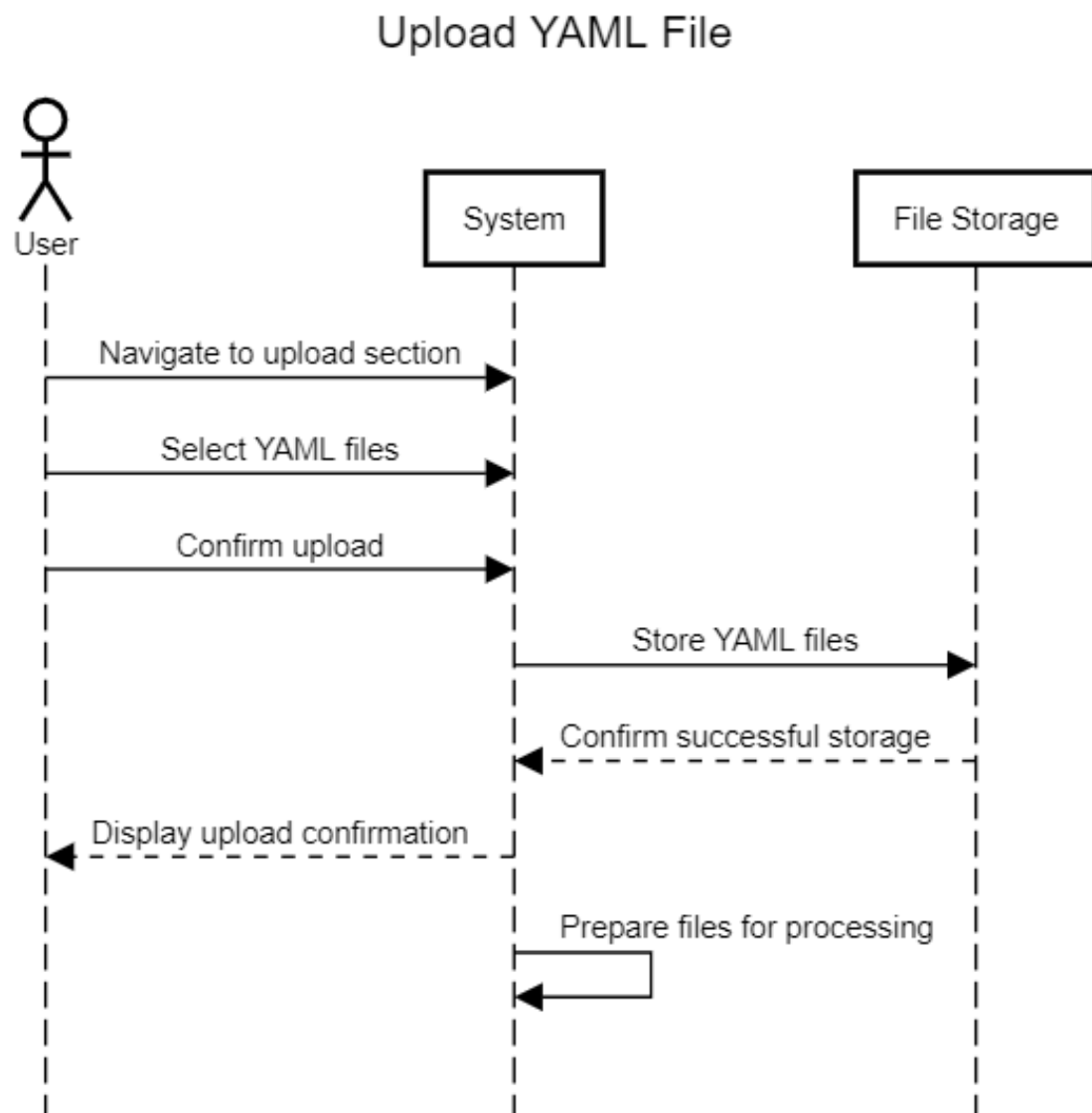


Figure 3.8: Upload Yaml File

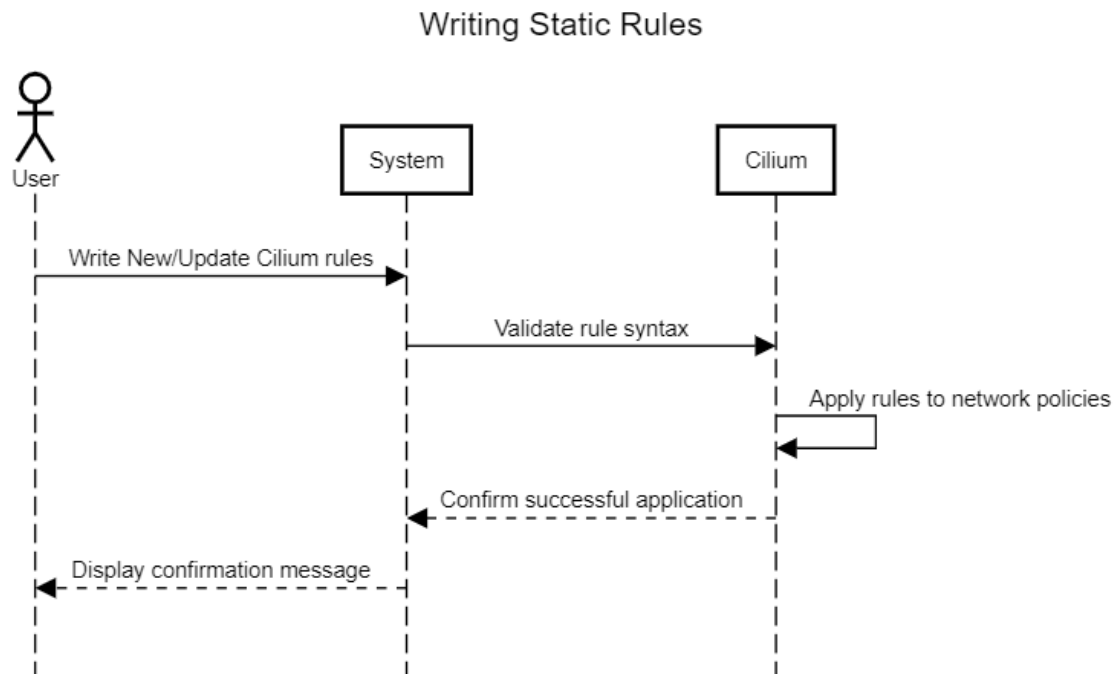


Figure 3.9: Writing Static Rules

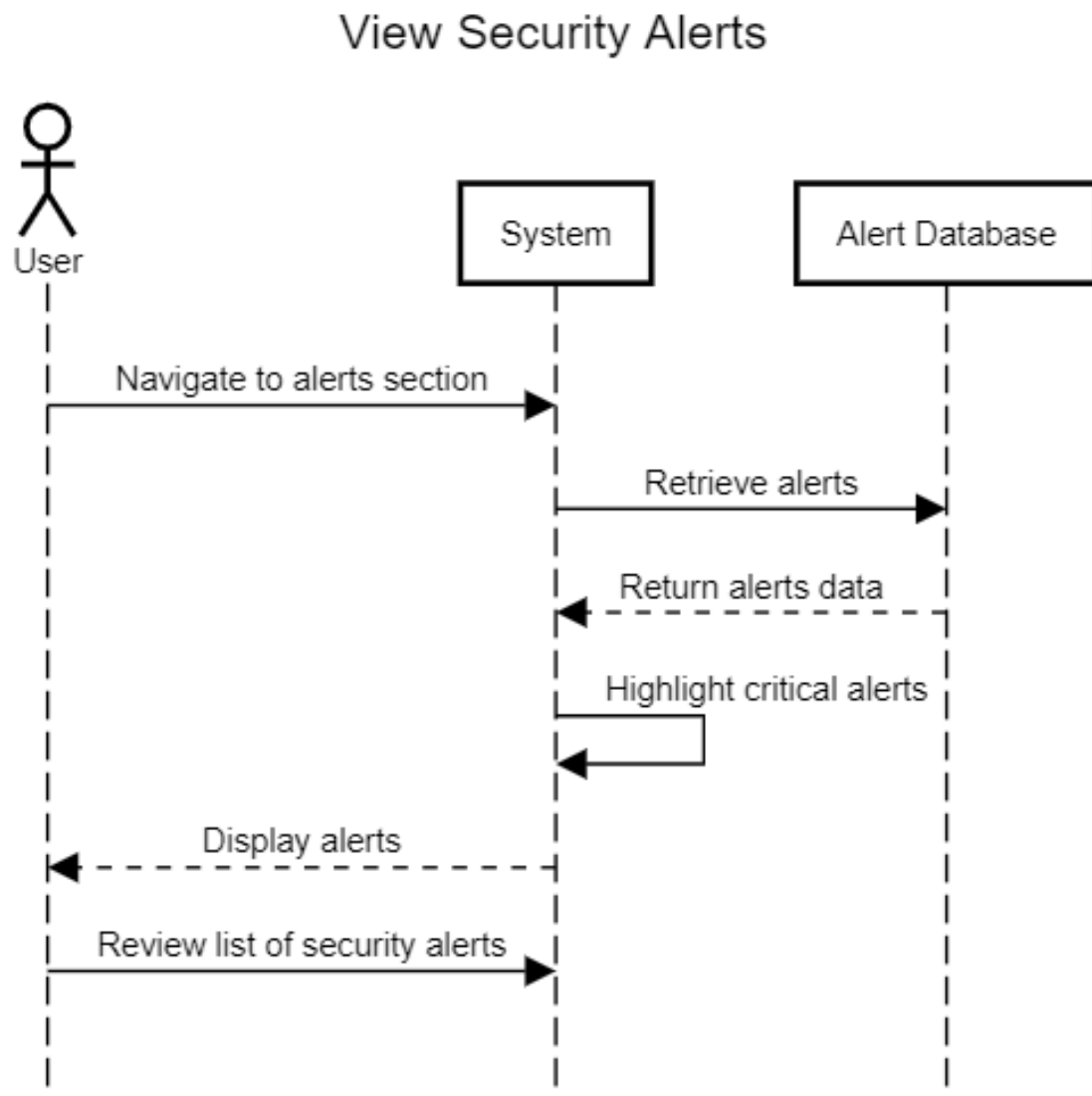


Figure 3.10: View Security Alert

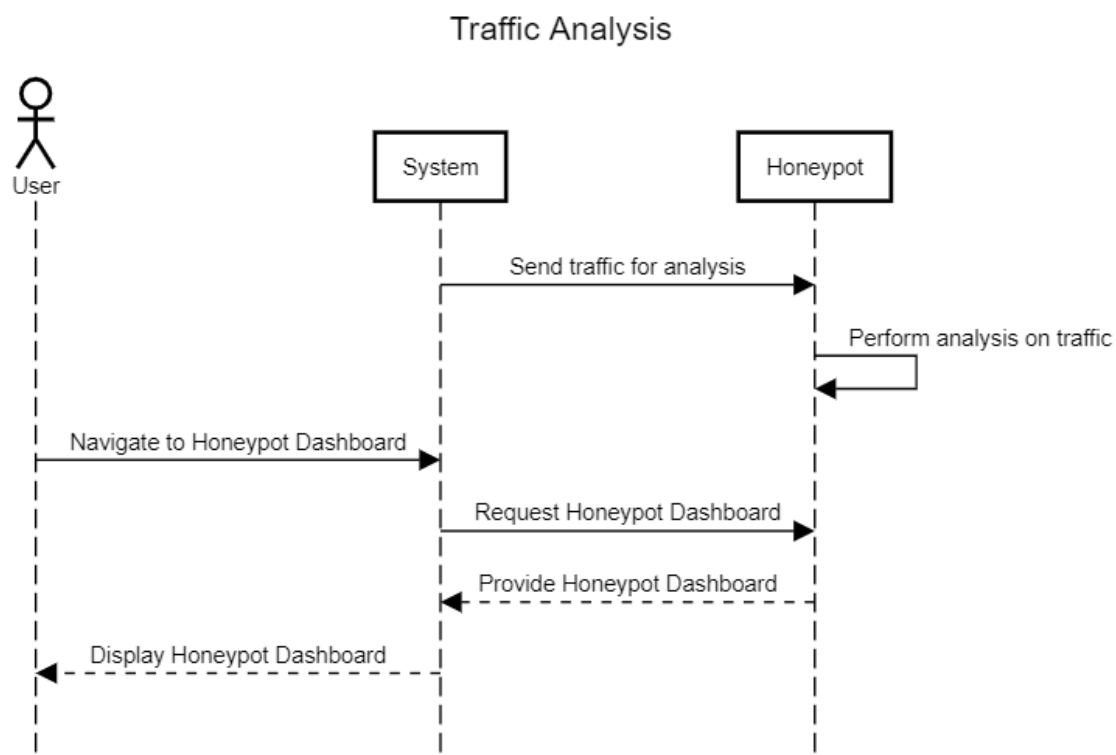


Figure 3.11: Traffic Analysis

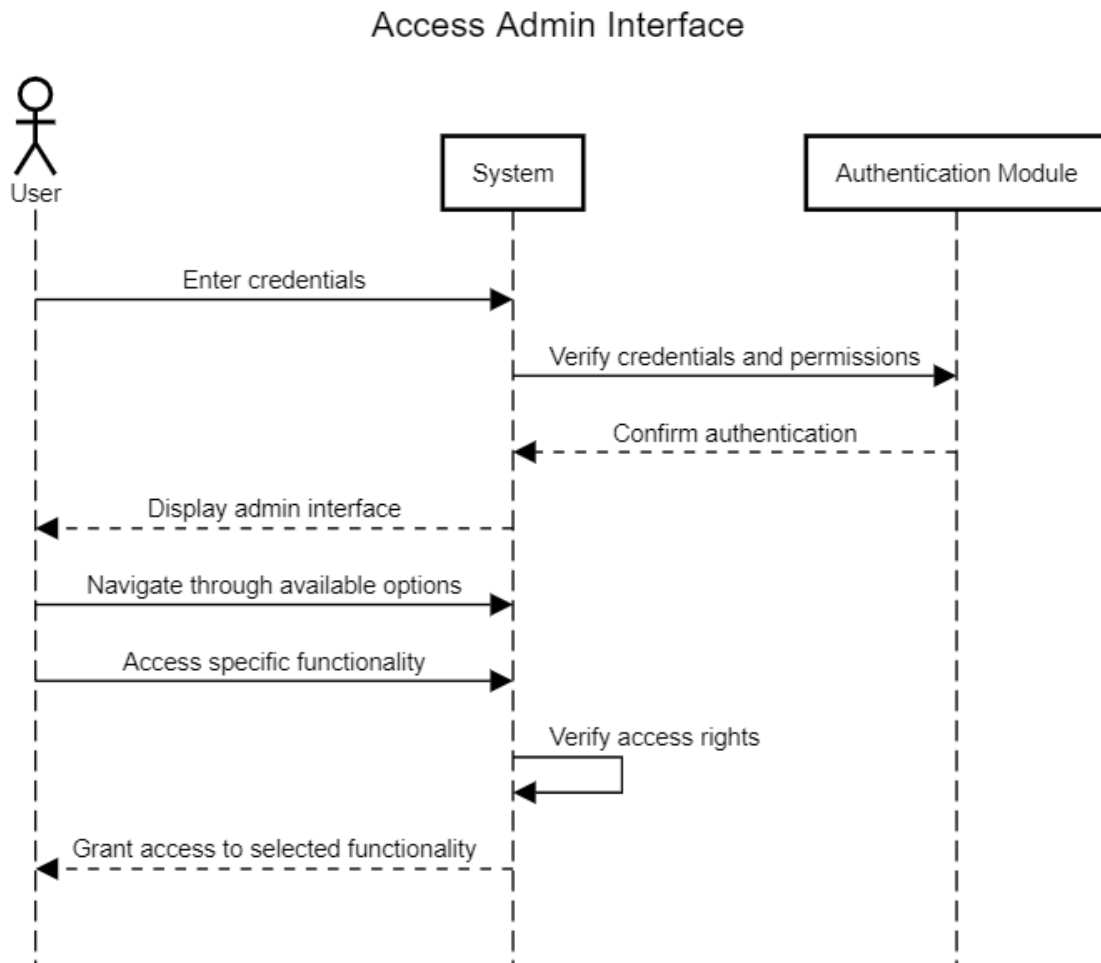


Figure 3.12: Access Admin Interface

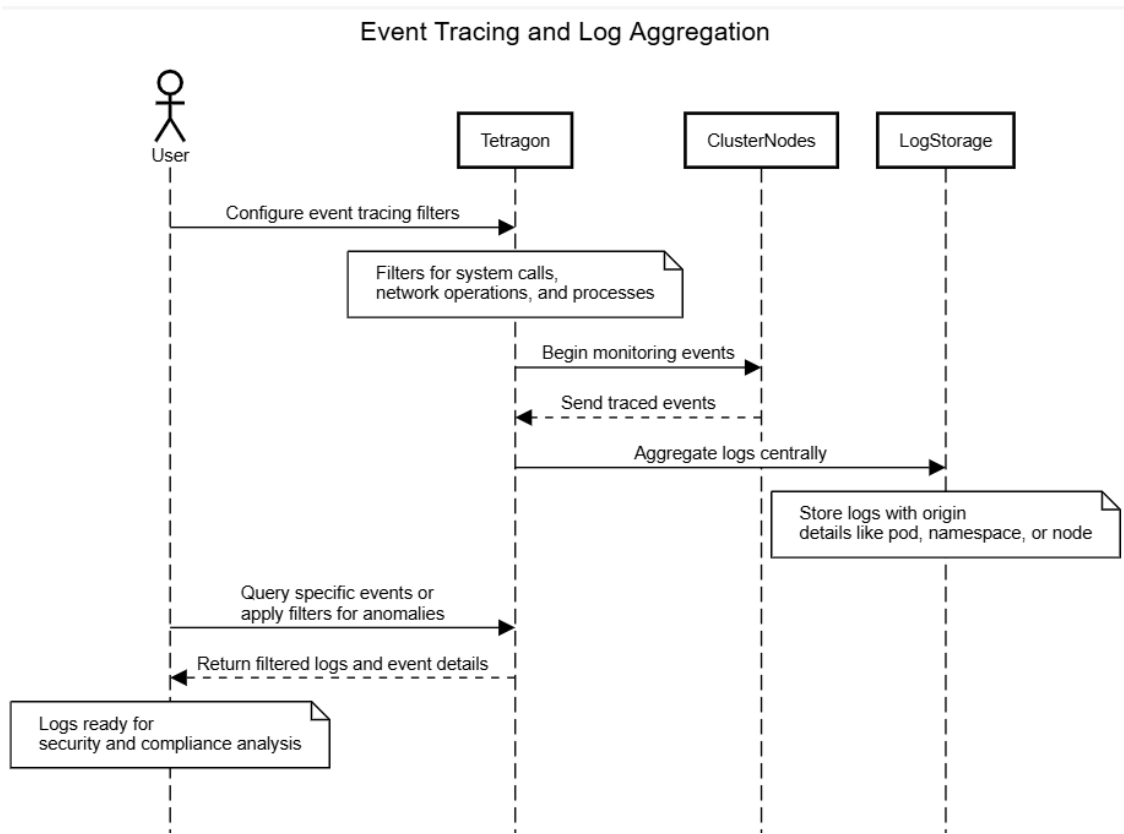


Figure 3.13: Log Aggregation and Event Tracing using Tetragon

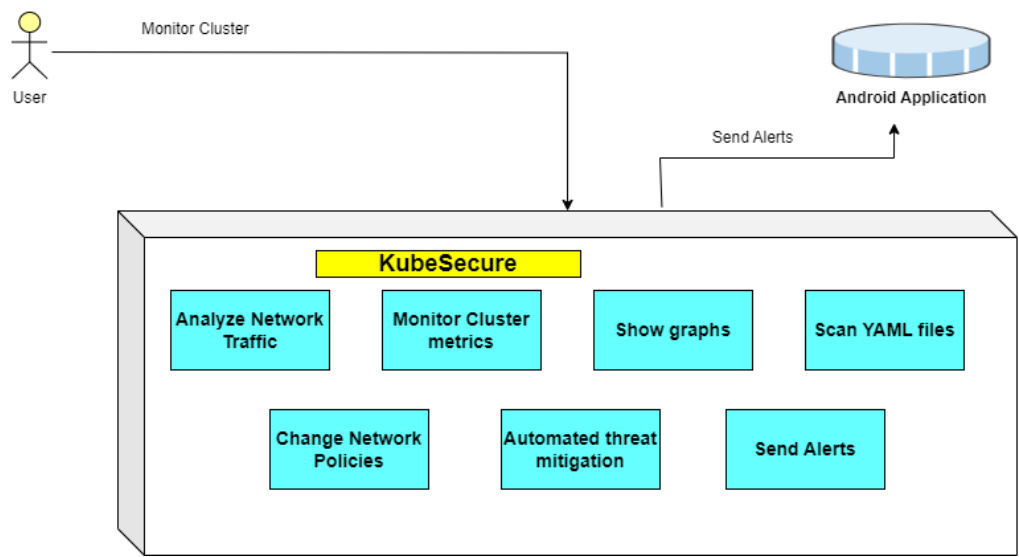


Figure 3.14: Data Flow Level 0

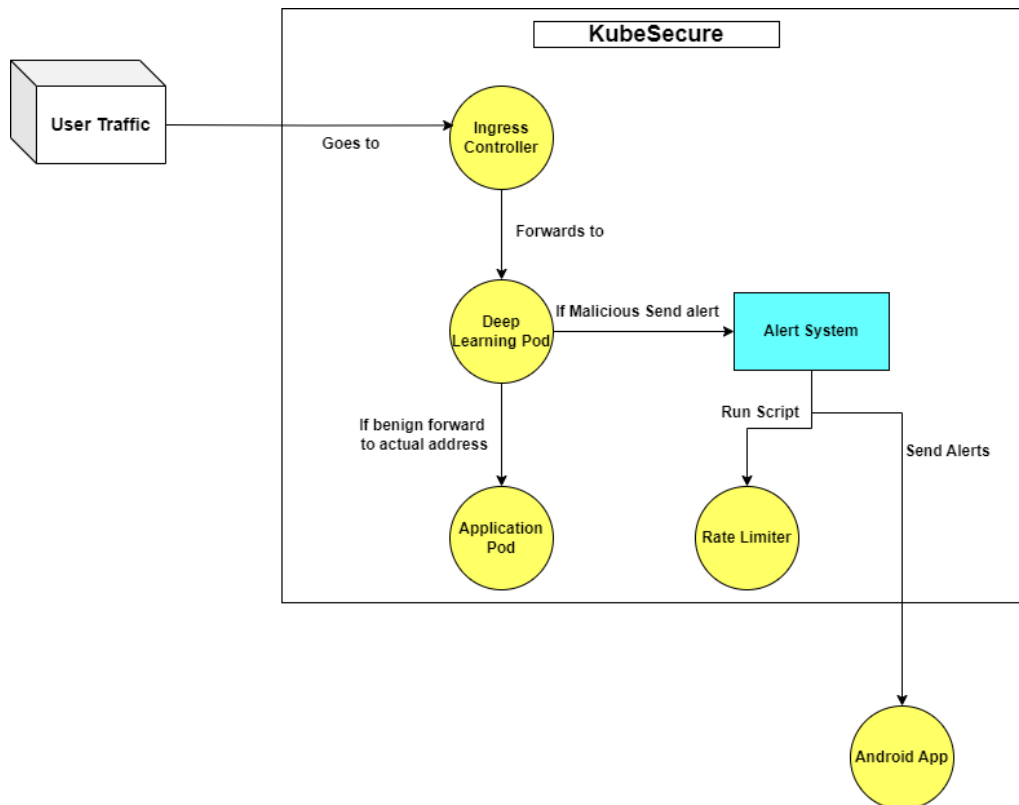


Figure 3.15: Data Flow Level 1

Chapter 4

Implementation and Testing

4.1 Algorithm Design

4.1.1 Setting up Multi-Node Cluster

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4

# 1 control plane node and 3 workers
nodes:
# the control plane node config
- role: control-plane
# the three workers
- role: worker
- role: worker
```

Figure 4.1: Yaml file for Multi Node Cluster

4.1.2 Traffic Mirroring

Algorithm 1: Real-Time Flow Data Capture and Display

Input: Network interface (ens33) for traffic monitoring, output file flows.csv

Output: Continuously printed DataFrame with traffic flow data

```
Function run_cicflowmeter():
    Start the subprocess to execute `sudo cicflowmeter -i ens33 -c flows.csv`
    Return the process object
End Function

Function stop_cicflowmeter(process):
    Send SIGINT signal to gracefully stop the subprocess
End Function

Function read_and_print_csv(file_path="flows.csv"):
    Try:
        Read the CSV file into a pandas DataFrame
        Print the DataFrame
    Catch any exception and print an error message
End Function

Function main():
    While True do:
        process ← run_cicflowmeter()
        collect data for 3 seconds
        stop_cicflowmeter(process)
        Wait for 1 second
        read_and_print_csv("flows.csv")
    End While
End Function

Call main()
```

Figure 4.2: Traffic Mirroring Algorithm

4.1.3 Data Preprocessing

Data Preprocessing Pipeline for Network Traffic Classification

Input: CSV files from benign and malicious folders

Output: Preprocessed, balanced, and feature-engineered dataset saved to a CSV file

```
Combine all CSV files from "benign" and "malicious" folders into separate DataFrames.
Merge the benign and malicious DataFrames into a single DataFrame (combined_df).
Shuffle combined_df and save to 'combined_shuffled_traffic.csv'.
```

```
Function preprocess_data(file_path):
    Load the dataset from file_path.
    Remove features with >90% zero values (optional based on user input).
    Handle missing values:
        - Fill numeric features with the median or use KNN imputation.
        - Fill categorical features with the mode.
    Clip numeric features to handle outliers.
    Scale numeric features using RobustScaler.
    Encode categorical features and the target column (if applicable).
    Engineer new features based on existing columns.
    Balance dataset classes using SMOTE if the target column exists.
    Return the preprocessed DataFrame.
```

```
Plot the correlation matrix for the preprocessed dataset using seaborn.
```

Figure 4.3: Data Preprocessing

4.1.4 Model Design

Network traffic typically exhibits both spatial patterns (e.g., correlations between features such as packet size, port usage, or protocol types) and temporal dependencies (e.g., sequential changes in traffic patterns over time). We chose to implement Convolution Long Short Term Memory model (CLSTM) because the **convolutional layers** in CLSTM extract spatial features by identifying local patterns and correlations between variables in the traffic data. The **LSTM layers** model temporal dependencies, capturing how traffic behavior evolves over time. This is critical for distinguishing between normal and anomalous traffic patterns.

4.1.4.1 Model Architecture

- **Convolutional Layers:** Extract spatial patterns in the input sequences with two Conv1D layers.
- **LSTM Layers:** Capture temporal dependencies in the data using two LSTM layers.
- **Dense Layers:** Perform final classification with fully connected layers and a sigmoid activation for binary output.

4.2 Testing Details

4.2.1 Alerting System

4.2.1.1 Prometheus Alerting Overview

Prometheus is the monitoring and alerting mechanism to monitor targets for status metrics based on the specified intervals and store them. This encompasses an enhanced alerting engine whereby the alerts generated are rule based on metrics data.

Key Components of Prometheus Alerting:

- **Alerting Rules:** The alerting rules are set directly in the Prometheus configuration file. An alert is defined there in terms of rules that outline times when the alert should be activated. Example: Set up an alarm if the CPU usage is above 80 per cent for longer than 5 minutes.
- **Alert manager:** Prometheus send out the alert and passing them to Alert manager for consolidation through processes like deduplication and grouping as well as routing. It can integrate with the notifications as a Slack, via email or via a simple web hook.

4.2.2 Grafana Alerting Overview

In addition to enriched alerting, Prometheus and Grafana integrate metric visualization as well as the definition of alerts on the dashboard.

- **Alert Rules in Grafana:** A user-specific alert can be generated on graphs or panels with set thresholds and conditions. These rules are checked and re-checked by Grafana.
- **Notification Channels:** The Grafana application supports sending of alerts through email, Slack, and web-hooks as channels.

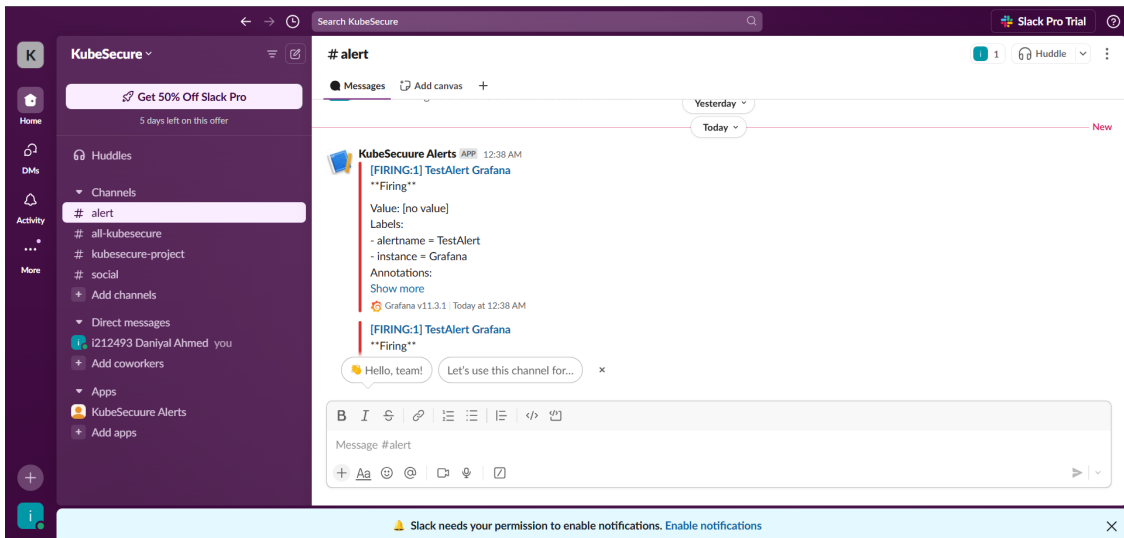


Figure 4.4: Grafana Slack Alert

4.2.3 Tetragon Tracing Policies

Tetragon is an open-source eBPF-based security observability tool developed by Cilium. It monitors and enforces runtime behavior in Kubernetes and Linux environments by tracing system calls, network events, and process activity.

Tetragon uses flexible tracing policies that allow users to define specific events to monitor, such as file access, process execution, or network connections. These policies enable fine-grained visibility and proactive security responses by automatically detecting suspicious or unauthorized behaviors.

4.2.3.1 Blocking mkdir command to restrict directory making

```
apiVersion: "cilium.io/v1alpha1"
kind: TracingPolicy
metadata:
  name: "block-writing"
spec:
  kprobes:
    - call: "sys_execve"
      syscall: true
      args:
        - index: 0
          type: "string"
  selectors:
```

```

- matchArgs:
  - index: 0
    operator: "Equal"
    values:
      - "/usr/bin/mkdir"
  matchActions:
    - action: Sigkill

```

The YAML file above checks for EBPf kernel system call `sys_execve`. If the call contains the path to `/usr/bin/mkdir`. It it send the signal to the kernel to kill the command as shown in figure 4.5.

```

hani@pcn-System-Product-Name:~$ kubectl get tracingpolicies
NAME                                AGE
blockdirectorymaking               14d
hani@pcn-System-Product-Name:~$ kubectl exec -it backend-575585967-6qgm9 -- /bin/bash
root@backend-575585967-6qgm9:/app# mkdir
Killed
root@backend-575585967-6qgm9:/app#

```

Figure 4.5: Block mkdir using Tetragon

Using different tracing policies, the user can specify which commands to block to restrict pod level attacks. Moreover, instead of writing and applying whole YAML file we have provided the functionality in the website to write the name of the command and block it as shown in figure

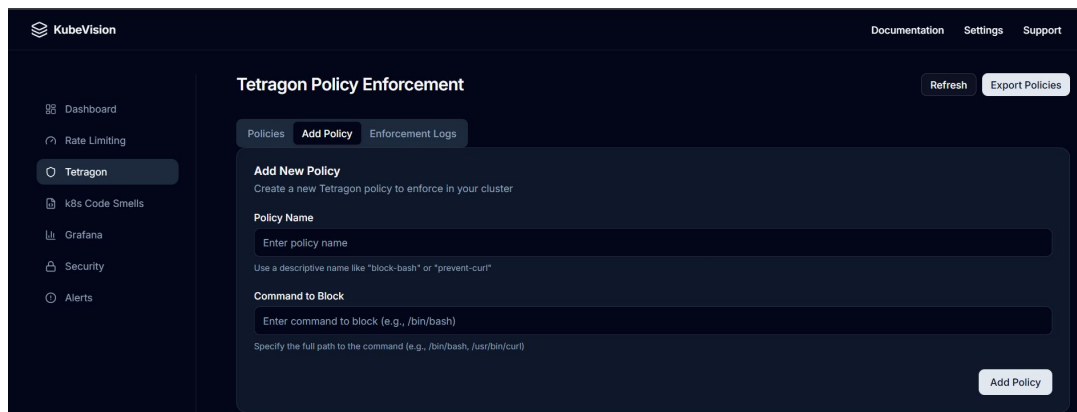


Figure 4.6: Enter command to block

4.2.4 Yaml File Security Misconfiguration

Certain rules have been implemented in our website to identify some critical security misconfigurations. For example: missing resource limitations, running pod on root level

etc. User can upload a yaml file to check for the security misconfiguration. For testing the yaml file below was used and the output is in figure

```

apiVersion: v1
kind: Pod
metadata:
  name: insecure-pod
  namespace: default
spec:
  containers:
    - name: insecure-container
      image: nginx:latest # Hardcoded latest tag
      securityContext:
        privileged: true # Overprivileged Pod
        runAsUser: 0 # Running as root
        allowPrivilegeEscalation: true # Privilege escalation allowed
      resources: {} # Missing resource requests and limits
      livenessProbe: # Missing readinessProbe
      httpGet:
        path: /
        port: 80
  volumes:
    - name: insecure-volume
      hostPath: # Using hostPath, security risk
        path: /etc/password
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: high-replica-deployment
  namespace: default
spec:
  replicas: 150 # Excessive replicas
  selector:
    matchLabels:
      app: test
  template:
    metadata:
      labels:
        app: test

```

```
spec:
  containers:
    - name: app-container
      image: myapp:v1
      securityContext: {} # Missing security context
      resources:
        limits:
          cpu: "1"
          memory: "512Mi"
        requests:
          cpu: "0.5"
          memory: "256Mi"

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: large-configmap
  namespace: test
data:
  key1: value1
  key2: value2
  key3: value3
  # Adding more than 100 entries (not fully listed here)
  key100: value100
  key101: value101 # Large ConfigMap detection

---
apiVersion: v1
kind: Secret
metadata:
  name: plain-text-secret
  namespace: default
data:
  password: "c3VwZXJzZWNyZXQ=" # Base64-encoded, but still a large secret
  key1: "bG9uZ3ZhbHVl" # Large plain text secret

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
```

```

metadata:
  name: wildcard-clusterrolebinding
subjects:
  - kind: Group
    name: system:authenticated # Wildcard RBAC binding
roleRef:
  kind: ClusterRole
  name: cluster-admin
  apiGroup: rbac.authorization.k8s.io

```

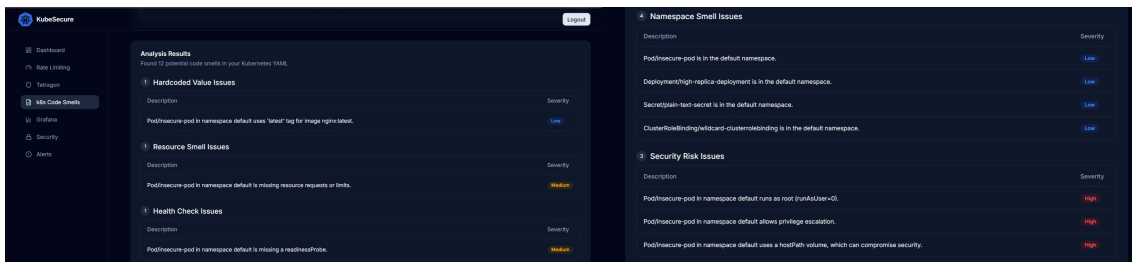


Figure 4.7: Security Smells in Yaml file

4.2.5 Unit Testing

Unit testing plays an important role in making sure each part of the KubeSecure system works correctly on its own. This section explains how we tested the main parts of the system. We checked how they perform in different situations—like when handling real network traffic, spotting unusual activity, applying rate limits, and sending alerts. All tests were done with KubeSecure running in a Kubernetes cluster with NGINX. The goal was to check that key features like traffic monitoring, anomaly detection, and policy enforcement were working as expected.

We started by preparing a DoS (Denial of Service) attack dataset and captured real traffic using the `server.py` script. The system then processed this traffic to spot any strange behavior, applied rate limits when needed, and sent alerts through Slack and the web interface. Later, we tested if the rate limits could be added and removed correctly, using Locust to simulate traffic and checking the results through the KubeSecure web interface. We also tested unusual cases to make sure the system could handle them well.

4.2.5.1 Data Preparation and Traffic Capture

Setting up the DoS attack data and capturing real-time traffic.

In the first test, we checked if the system could handle a DoS attack dataset and capture traffic from a Kubernetes app running behind NGINX. We ran the `server.py` script, which started the `cicflowmeter` to capture traffic every 3 seconds.

- *Test Case 1:* Make sure the dataset loads and traffic capture starts correctly.
- *Results:* The system loaded the dataset and started capturing traffic without any problems.

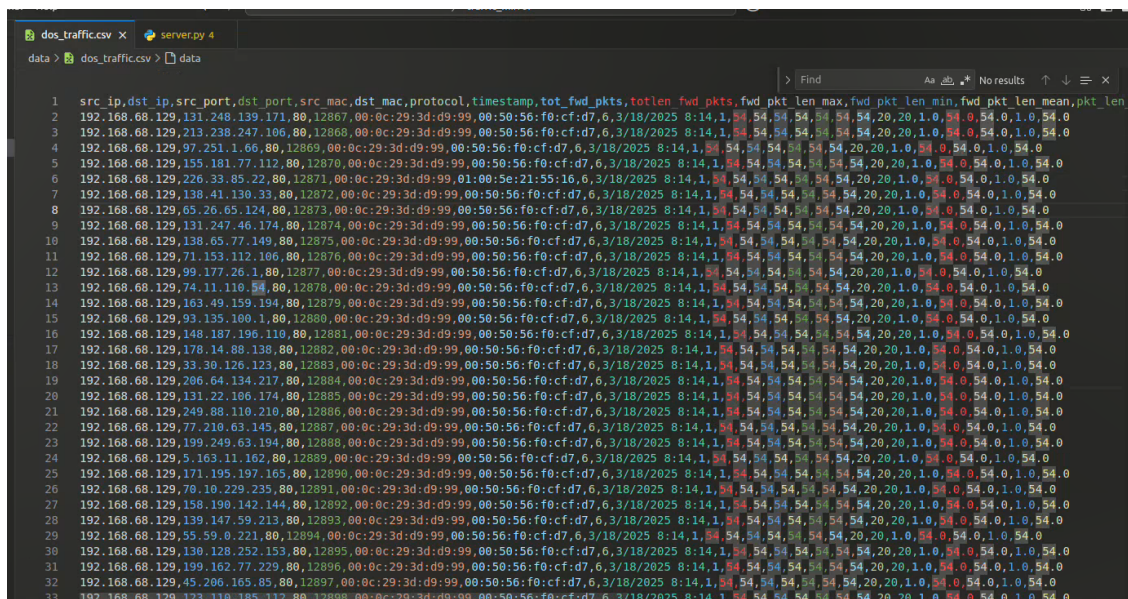


Figure 4.8: DoS attack dataset used in testing, showing sample data.

```
server.py > main
265
266
267
268
269 def main():
270     try:
271
272         while True:
273             # Start cicflowmeter
274             process = run_cicflowmeter()
275             print("Capture Starting\n")
276             # Let it capture traffic for 3 seconds
277             time.sleep(3)
278             print("Capture Stopping\n")
279             # Stop the cicflowmeter process
280             stop_cicflowmeter(process)
281
282             # Give a small delay to ensure the file is fully written
283             time.sleep(2)
284
285             # Read and print the DataFrame from flows.csv
286             df = read_and_print_csv("./data/dos_traffic.csv")
287             predict(df)
288             time.sleep(5)
289         except KeyboardInterrupt:
290             print("Exiting")
291
292
293 # Run the main function
294 if __name__ == "__main__":
295     main()
296
297
```

Find No results

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

2025-04-11 13:03:08.391563: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
ns.
To enable the following instructions: AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
W0808 08:08:1744358589.928662 59899 gpu_device.cc:2341] Cannot dlopen some GPU libraries. Please make sure the missing libraries mentioned above are installed properly if you would
like to use GPU. Follow the guide at <https://www.tensorflow.org/install/gpu> for how to download and setup the required libraries for your platform.
Skipping registering GPU devices...
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
Capture Starting
Capture Stopping

Figure 4.9: Traffic capture output from `server.py`, showing traffic flowing through NGINX.

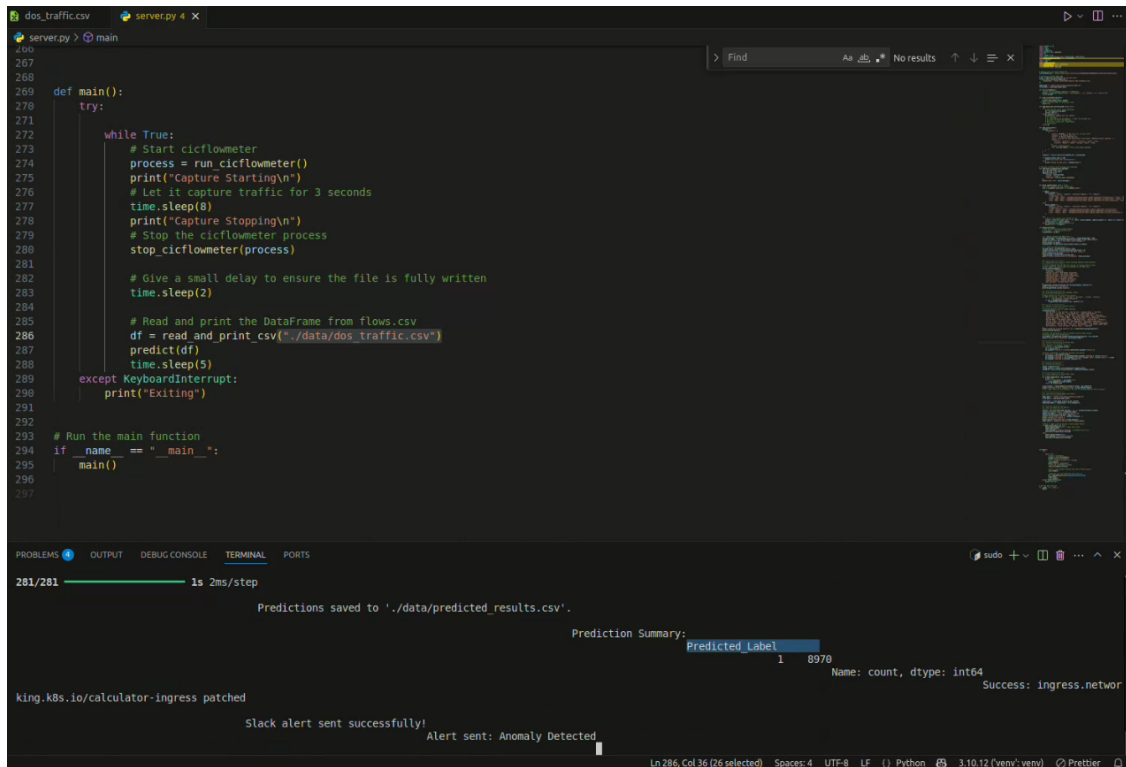
4.2.5.2 Anomaly Detection and Alerting

Spotting unusual activity and sending alerts to Slack and the web interface.

This test focused on checking if the system could detect strange or suspicious activity in the traffic and send alerts. The system successfully found such activity and sent alerts through Slack and the KubeSecure interface.

- *Test Case 2:* Make sure the system detects anomalies and sends alerts.
- *Results:* Anomalies were detected, and alerts were successfully sent to Slack and the web interface.

4. Implementation and Testing



```
server.py > main
266
267
268
269 def main():
270     try:
271
272         while True:
273             # Start cicflowmeter
274             process = run_cicflowmeter()
275             print("Capture Starting\n")
276             # Let it capture traffic for 3 seconds
277             time.sleep(3)
278             print("Capture Stopping\n")
279             # Stop the cicflowmeter process
280             stop_cicflowmeter(process)
281
282             # Give a small delay to ensure the file is fully written
283             time.sleep(2)
284
285             # Read and print the DataFrame from flows.csv
286             df = read_and_print_csv("../data/dos_traffic.csv")
287             predict(df)
288             time.sleep(5)
289         except KeyboardInterrupt:
290             print("Exiting")
291
292
293 # Run the main function
294 if __name__ == "__main__":
295     main()
296
297
281/281 1s 2ms/step
Predictions saved to '../data/predicted_results.csv'.
Prediction Summary:
Predicted_Label
1 8970
Name: count, dtype: int64
Success: Ingress.network
king.k8s.io/calculator-ingress patched
Slack alert sent successfully!
Alert sent: Anomaly Detected
```

Figure 4.10: Terminal showing that an anomaly was detected and a Slack alert was sent.

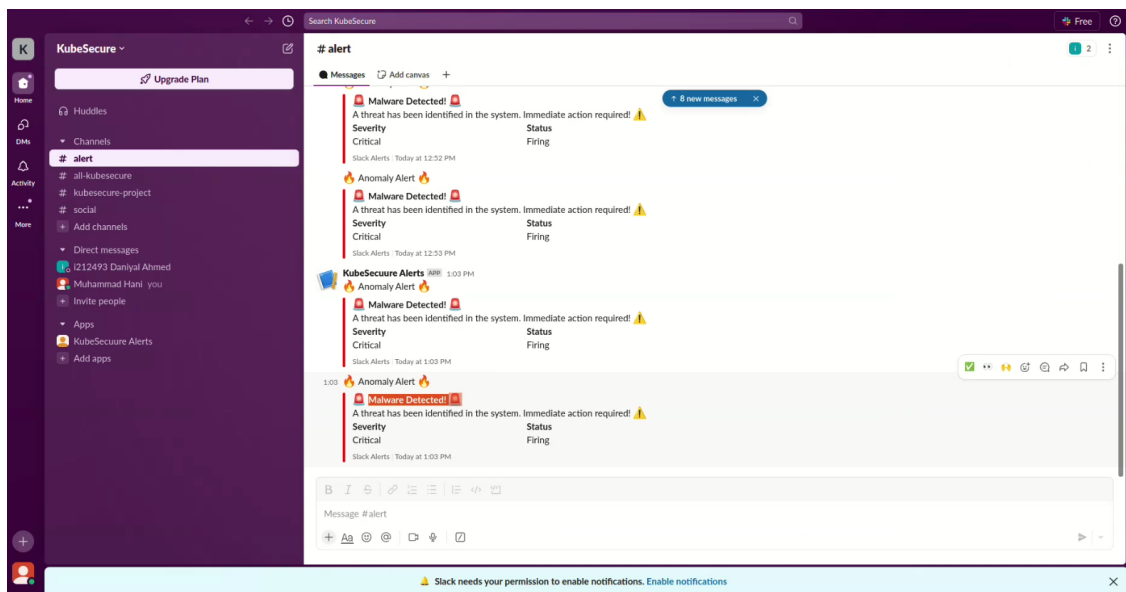


Figure 4.11: Slack alert confirming that the anomaly was detected.

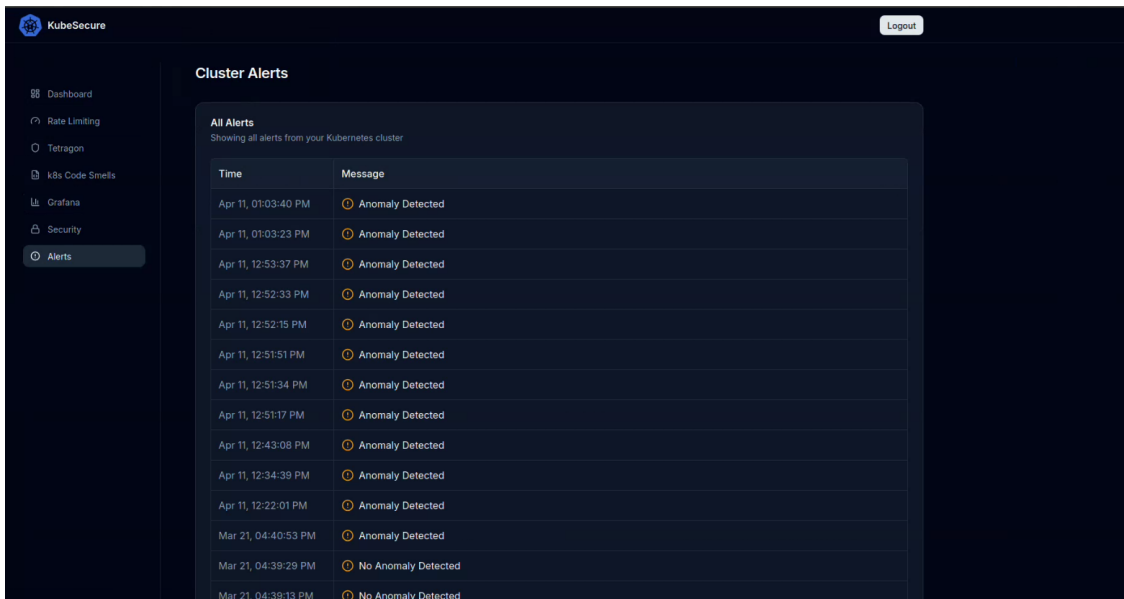


Figure 4.12: Web interface showing multiple alerts from detected anomalies.

4.2.5.3 Rate Limiting Application

Applying rate limits on the Kubernetes API server.

Here, we tested if the system could limit how many requests were allowed per second. We set a limit of 10 requests per second using the KubeSecure web interface. Then we used Locust to simulate traffic. The results showed that the system was correctly enforcing the rate limit.

- *Test Case 3:* Check if rate limiting works with a 10 req/s limit.
- *Results:* The limit was applied correctly, and Locust showed that 91% of requests failed due to the limit.

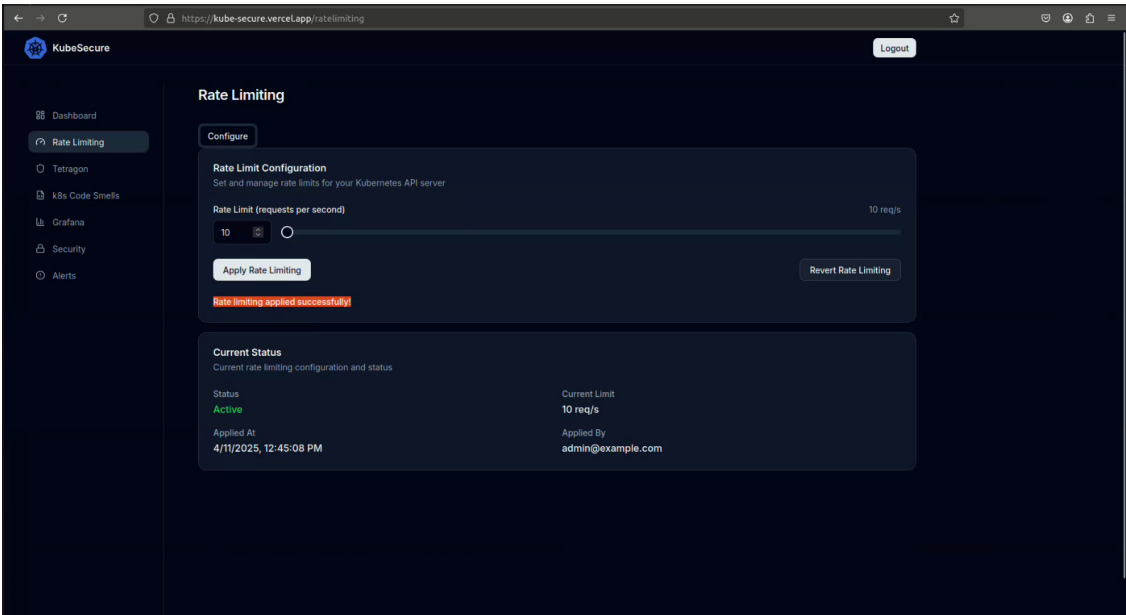


Figure 4.13: Web interface confirming that rate limiting was applied.

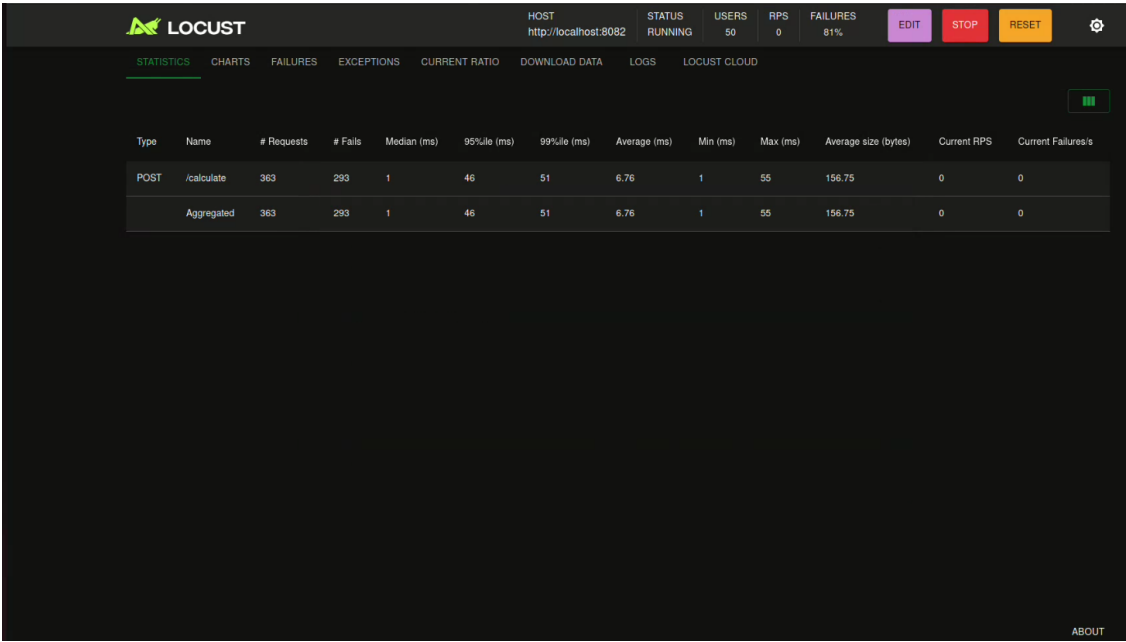


Figure 4.14: Locust report showing 91% of requests failed due to rate limiting.

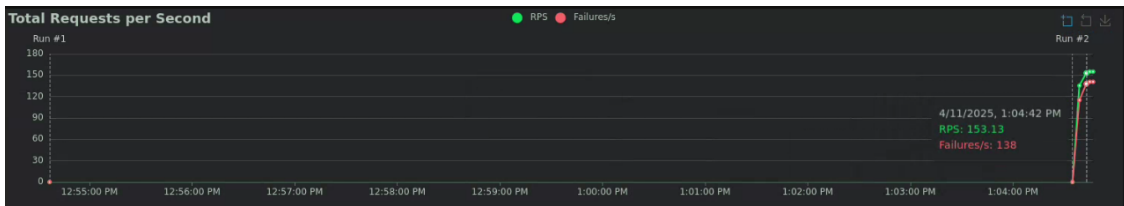


Figure 4.15: Graph from Locust showing 138 out of 153 requests failed, confirming rate limiting.

4.2.5.4 Rate Limiting Reversion

Removing rate limits and checking traffic success.

The final test checked if we could remove the rate limit from the web interface and whether traffic returned to normal. After removing the limit, all traffic requests were successful.

- *Test Case 4:* Make sure rate limiting can be removed and traffic works fine again.
- *Results:* The limit was successfully removed, and Locust showed 0% request failures with 155 successful requests.

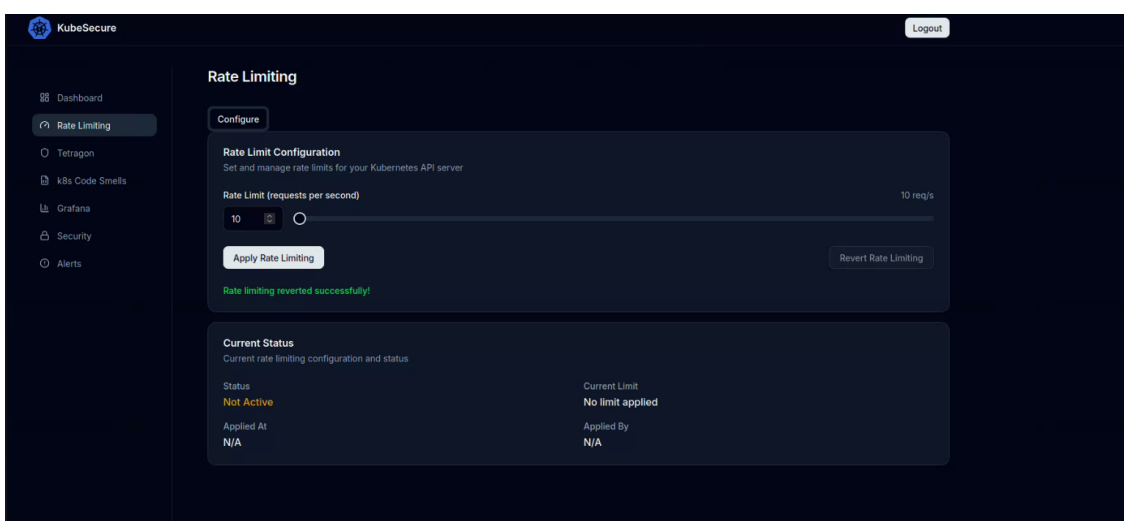


Figure 4.16: Web interface confirming that the rate limit was removed.

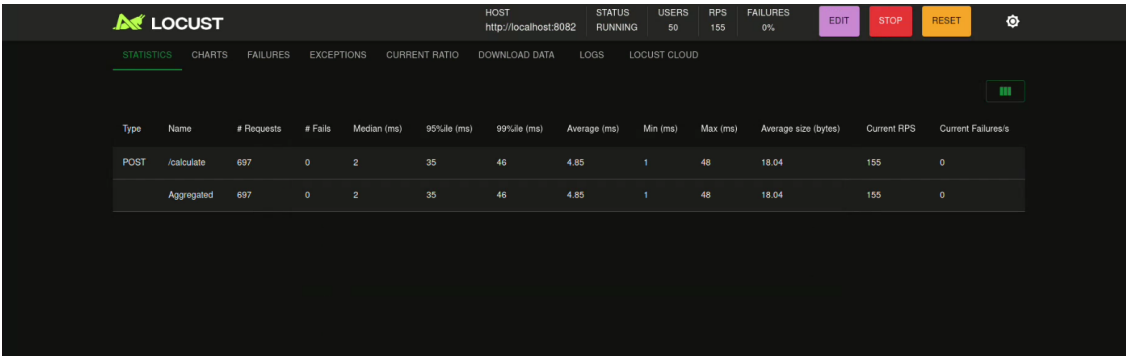


Figure 4.17: Locust report showing 0% failures after the rate limit was removed.

In short, these tests proved that the KubeSecure system can reliably capture live traffic, detect unusual activity, apply and remove rate limits, and send alerts. All system parts worked well together during testing, showing that KubeSecure is ready for real use.

Chapter 5

Conclusions and Future Work

5.1 Conclusions

The KubeSecure system has emerged as a strong candidate for improving the security of applications running in Kubernetes clusters after large amounts of development and testing. The project also successfully demonstrated the feasibility of creating live traffic captures using the `server.py` script and `cicflowmeter` process to the processing of data from applications being managed by NGINX. The anomaly detection module performed well during the live tests returning notices of detected malicious traffic patterns and then using those suspicious patterns to send alerts to Slack and the web interface, thus creating a visible and timely response system.

The implementation of rate limiting was in fact a highlighter when testing using Locust showed (and confirmed) successful mitigation of potential denial-of-service (DoS) attacks. The KubeSecure system enforced a limitation on 10 requests per second, thus minimizing the severity of simulated attacks on its applications. The ability to override the 10 requests per second limit again from the web interface showed how user-friendly this web-based KubeSecure system is designed to be. The reliable performance of the KubeSecure system is highlighted by its zero failure rate after removing the rate limit on external web requests

This entire project has established a sound footing for securing Kubernetes-based deployments. The combination of traffic monitoring, anomaly detection, and dynamic rate limiting, with real-time alerts, has provided some good protection against many typical security threats. It has been gratifying to watch how these components combine to produce a functional and usable security tool, and we pleased with how far we've come tackling the specific problems introduced by contemporary cloud environments.

5.2 Future Work

There is ample opportunity to expand the project and enhance KubeSecure in the future. An exciting area for development would be adding additional mitigation methods. For example, we could implement an intrusion prevention system block IP pools, using deep learning techniques to analyze traffic both for abnormal patterns and then shift the security policy in response to a new pattern that is evolving, which would represent a substantial advancement beyond just detection into the realm of actual physical remediation of an incident during an attack.

Moreover, logs are very important to debug microservices issues and get a better understanding to optimize the performance of an application and detect anomalies. We can finetune an LLM to detect different types of anomalies in logs.

Another area to look at is how to improve the existing anomaly detection algorithm. Maybe we could add better features or better datasets to identify even more subtle attacks we have not yet detected. There are also many ways to further improve the alerting system, such as email notifications or integration with additional collaboration applications beyond Slack to allow for even greater distribution.

Testing KubeSecure in large, more complex Kubernetes clusters will help inform us about its scalability limitations and performance. Additionally, we could investigate implementing a user-friendly dashboard to provide more visibility to system administrators over the status of KubeSecure and allow for storage of historical data, which will enable more effective management of security in a real-world environment.

This project has been a wonderful springboard, and we can't wait to continue expanding the possibilities of what KubeSecure can do! There is so much more to develop, and we look forward to seeing what we create next!

Bibliography

- [1] Cyber attacks kubernetes. <https://purplesec.us/breach-report/kubernetes-clusters-hacked/>. Accessed: 2024-09-07.
- [2] Distributed denial of service attack. <https://www.cloudflare.com/learning/ddos/what-is-a-ddos-attack/>. Accessed: 2024-09-07.
- [3] Kubernetes. <https://kubernetes.io/docs/home/>. Accessed: 2024-09-04.
- [4] Loss due to cyber attacks. <https://www.statista.com/chart/32341/worldwide-reported-losses-connected-to-cybercrime/>. Accessed: 2024-09-04.
- [5] Josue Genaro Almaraz-Rivera. An anomaly-based detection system for monitoring kubernetes infrastructures. *IEEE Latin America Transactions*, 21(3):457–465, 2023.
- [6] Tae-Young Kim and Sung-Bae Cho. Web traffic anomaly detection using c-lstm neural networks. *Expert Systems with Applications*, 106:66–76, 2018.